

 DISTRIBUTED JOB SCHEDULER

Deliverables & Documentation

A production-grade distributed job scheduler — comparable to Temporal, BullMQ, Sidekiq, and Celery — built for reliability, horizontal scalability, and enterprise observability.

 Repository: <https://github.com/moneshs23/Job-Scheduler>

 Generated: July 5, 2026

Stack: FastAPI · Python 3.12 · SQLAlchemy 2.0 (async) · PostgreSQL · Redis
(Streams / Pub-Sub) · APScheduler · React · TypeScript · Vite · TailwindCSS · Docker
Compose

Table of Contents

Deliverables Checklist

[Jump to deliverables checklist →](#)

Documentation

Project Overview

- [Status](#)
- [Quick Links](#)
- [Tech Stack](#)
- [Architecture Overview](#)
- [Quickstart](#)
- [License](#)

Architecture

- [Table of Contents](#)
- [1. High-Level Architecture](#)
- [2. Microservice Architecture](#)
- [3. Folder Structure](#)
- [4. Database ER Diagram](#)
- [5. Sequence Diagrams](#)
- [6. Worker Lifecycle Diagram](#)
- [7. Job Lifecycle Diagram](#)
- [8. Queue Flow Diagram](#)
- [9. Deployment Diagram](#)
- [10. Component Rationale](#)
- [11. Frontend Design — Neobrutalism Theme](#)
- [12. Technology Decisions Summary](#)
- [Next Steps](#)

Database & ER Diagram

- [Tables](#)
- [Why Postgres, not just Redis](#)
- [The claim query](#)
- [Indexes worth knowing about](#)
- [Cascade rules](#)
- [Recurring / cron jobs](#)

API Documentation

- [Authentication](#)
- [Endpoints by resource](#)
- [Pagination, filtering, sorting](#)

- [Creating a job](#)
- [Errors](#)

Design Decisions

- [Postgres as the queue, Redis as a doorbell](#)
- [SELECT ... FOR UPDATE SKIP LOCKED](#) instead of a distributed lock
- [job.maxattempts](#) as the retry cap, not [retrypolicy.maxretries](#)
- [serverdefault=func.now\(\)](#), never [serverdefault="now\(\)"](#)
- [Cron/recurring jobs](#) clone instances rather than templating at execution time
- [Workers](#) are scoped to one project
- [Leader election](#) for the scheduler, not for workers
- [Neobrutalism](#) for the dashboard

Deployment Guide

- [Local development \(no Docker\)](#)
- [Docker Compose](#)
- [Environment variables](#)
- [Migrations](#)
- [Production considerations](#) not yet wired up

Deliverables Checklist

Every item below is satisfied in this document and/or the linked source repository.

✓ Source code with setup instructions	In this PDF →
✓ Architecture diagram	In this PDF → · GitHub →
✓ ER diagram	In this PDF → · GitHub →
✓ API documentation	In this PDF → · GitHub →
✓ Design decisions document describing major trade-offs	In this PDF → · GitHub →
✓ Automated tests for critical functionality	In this PDF → · GitHub →

Automated tests: 19 passing pytest tests (unit + integration) in [backend/tests/](#), including a concurrency test that proves atomic job claiming (`SELECT ... FOR UPDATE SKIP LOCKED`) never lets two workers double-claim the same job.

Project Overview

[View on GitHub: README.md](#)

Distributed Job Scheduler

A production-grade distributed job scheduler — comparable to Temporal, BullMQ, Sidekiq, and Celery — built for reliability, horizontal scalability, and enterprise observability.

Status

Implemented and verified end-to-end. Backend (auth, orgs/projects, queues, jobs, workers, scheduler, DLQ, WebSocket realtime) and frontend (React dashboard, neobrutalism theme) both run locally against real Postgres/Redis, with 19 passing pytest tests (including a concurrency test proving atomic claiming never double-executes a job) and a Playwright-verified UI.

Quick Links

Document	Description
Architecture.md	High-level design, ER diagrams, sequence diagrams, deployment
Design- Decisions.md	Why Postgres over Redis-as-queue, SKIP LOCKED vs. distributed locks, retry semantics, leader election
Database.md	Schema, indexes, cascade rules, the claim query explained
API.md	Endpoint map, auth schemes, pagination — full schema lives at <code>/docs</code>

Document	Description
Deployment.md	Local dev setup, Docker Compose, environment variables

Tech Stack

- **Frontend:** React, TypeScript, Vite, TailwindCSS, React Query, React Router, Chart.js
- **Backend:** FastAPI, Python 3.12, SQLAlchemy 2.0 (async), APScheduler
- **Data:** PostgreSQL, Redis (Streams, Cache, Pub/Sub)
- **Ops:** Docker, Docker Compose, Prometheus, structured JSON logging

Architecture Overview

```
React Dashboard → FastAPI API → PostgreSQL (source of truth)
                  ↓               ↑
                APScheduler    Worker Pool (SKIP LOCKED claim)
                  ↓               ↑
                Redis Streams (dispatch notifications)
```

Quickstart

```
cp .env.example .env # set JWT_SECRET_KEY
docker compose up --build
```

Open <http://localhost:8080>, register, create a project, then start a worker for it — see [Deployment.md](#) for the full bootstrap sequence and local (non-Docker) setup.

License

MIT

Architecture

[View on GitHub: docs/Architecture.md](#)

Distributed Job Scheduler — Architecture

Status: Implemented. This document reflects the design as built — see the repo root README for what's running and verified.

A production-grade distributed job scheduler comparable to Temporal, BullMQ, Sidekiq, Celery, and cloud worker systems. Designed for millions of jobs, horizontal scale, atomic execution, and enterprise observability.

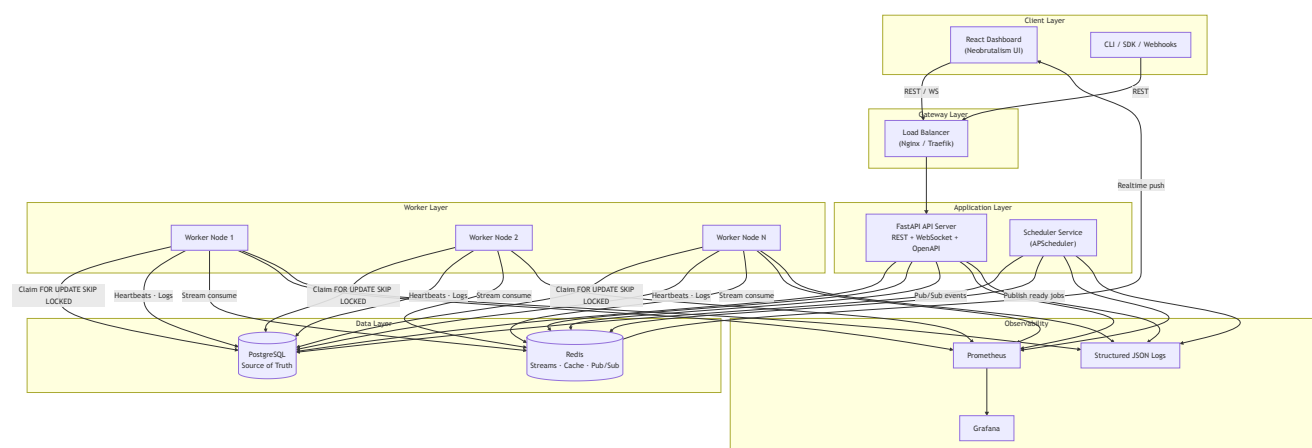
Table of Contents

1. [High-Level Architecture](#)
2. [Microservice Architecture](#)

3. [Folder Structure](#)
4. [Database ER Diagram](#)
5. [Sequence Diagrams](#)
6. [Worker Lifecycle](#)
7. [Job Lifecycle](#)
8. [Queue Flow](#)
9. [Deployment Diagram](#)
10. [Component Rationale](#)
11. [Frontend Design — Neobrutalism Theme](#)
12. [Technology Decisions Summary](#)

1. High-Level Architecture

The system follows a **modular monolith with horizontally scalable worker processes**. The API, scheduler, and workers share the same codebase but run as separate deployable processes — a pragmatic pattern used by Sidekiq (web + workers) and Celery (beat + workers + API).

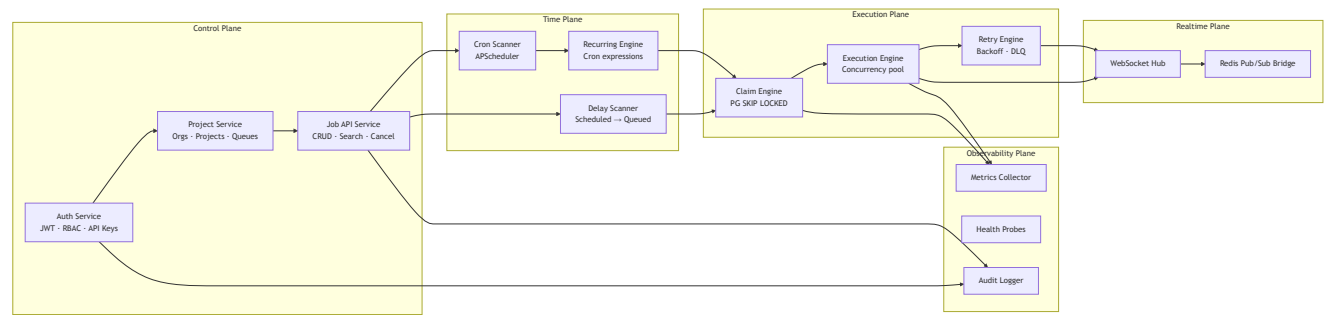


Design Principles

Principle	Implementation
Single source of truth	PostgreSQL holds all durable state (jobs, queues, workers, audit)
Atomic claiming	<code>SELECT ... FOR UPDATE SKIP LOCKED</code> inside a transaction
At-least-once delivery	Claim + idempotency keys + execution records prevent duplicate side effects
Separation of concerns	API (control plane) ≠ Scheduler (time plane) ≠ Workers (execution plane)
Fail-safe retries	Configurable backoff policies with DLQ escape hatch
Observable by default	Prometheus metrics, structured logs, WebSocket live feed

2. Microservice Architecture

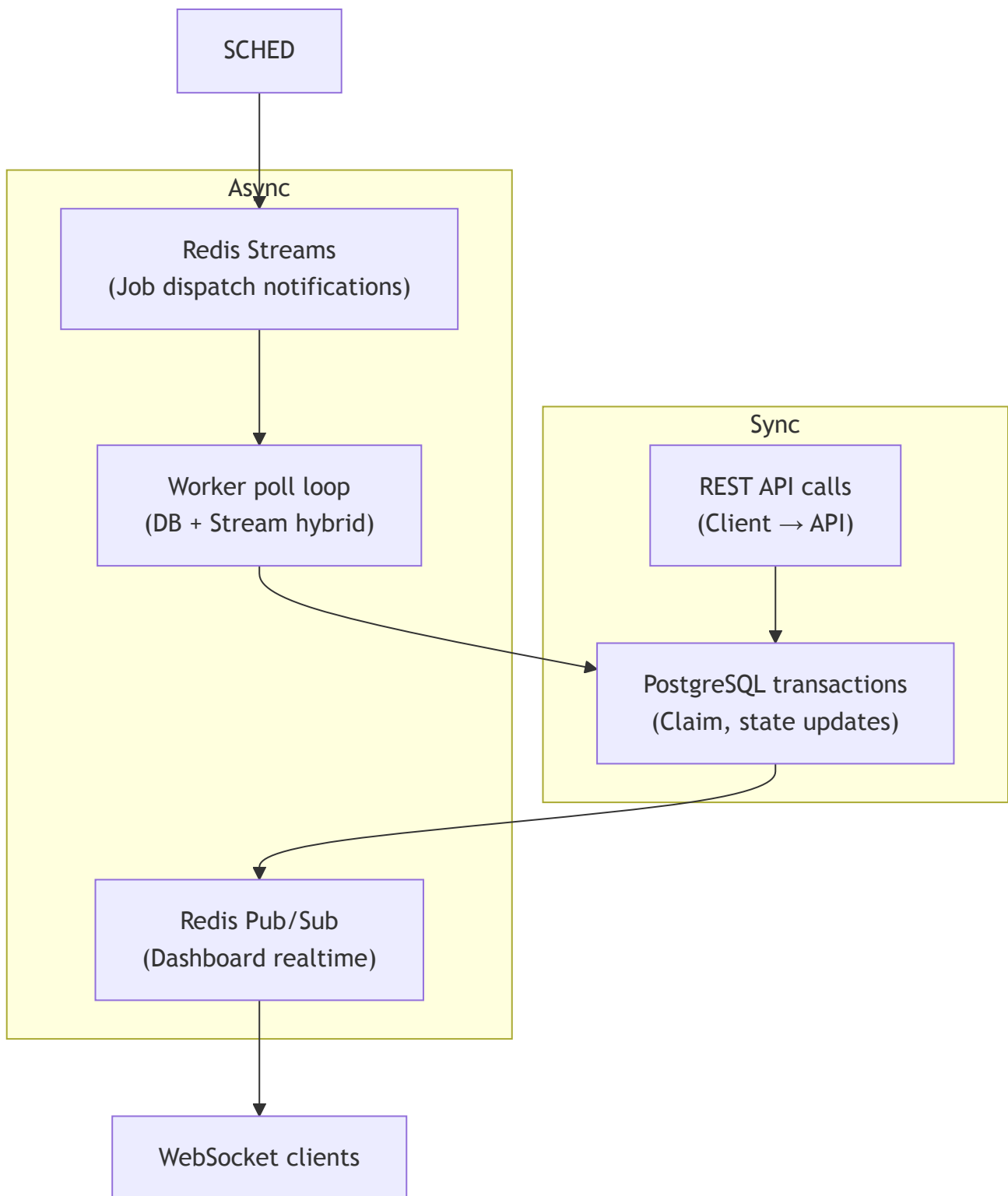
Although deployed from one repository, the runtime is split into **five logical services** that can be scaled independently.



Service Responsibilities

Service	Process	Scale Strategy
API Server	<code>uvicorn app.main:app</code>	Horizontal — stateless behind LB
Scheduler	<code>python -m app.scheduler.main</code>	Single leader with Redis distributed lock (optional HA standby)
Worker	<code>python -m app.workers.main</code>	Horizontal — add nodes per queue load
WebSocket	Embedded in API (or separate if needed)	Sticky sessions or Redis-backed fan-out
Metrics	<code>/metrics</code> endpoint on each process	Scraped by Prometheus

Inter-Service Communication



Hybrid dispatch model: The scheduler writes job state to PostgreSQL (durable) and publishes a lightweight notification to Redis Streams (fast wake-up). Workers primarily claim from PostgreSQL (correctness) and use Streams as a hint to reduce idle polling latency.

3. Folder Structure

This reflects the repository exactly as implemented (verified against the working tree, not the pre-implementation plan) — see github.com/moneshs23/Job-Scheduler.

```
distributed-job-scheduler/
├── README.md
├── docker-compose.yml
├── .env.example
├── .gitignore
├──
├── docs/
│   ├── Architecture.md      ← this document
│   ├── Design-Decisions.md
│   ├── Database.md
│   ├── API.md
│   └── Deployment.md
├──
├── docker/
│   └── prometheus/
│       └── prometheus.yml
├──
├── backend/
│   ├── pyproject.toml
│   ├── alembic.ini
│   ├── Dockerfile
│   ├── docker-entrypoint.sh
│   ├── migrations/
│   │   ├── env.py
│   │   └── versions/        # initial schema, queue unique constraint, now() default fix
│   ├── app/
│   │   ├── main.py          # FastAPI entrypoint
│   │   ├── config/
│   │   │   ├── settings.py  # Pydantic Settings (env-driven)
│   │   │   └── constants.py # job/worker status enums, Redis key prefixes
│   │   ├── api/
│   │   │   ├── router.py    # Aggregates all routers under /api/v1
│   │   │   └── routes/
│   │   │       ├── auth.py  # register/login/refresh/me/api-keys
│   │   │       ├── organizations.py # orgs, projects, audit-logs
│   │   │       ├── queues.py # queues + retry-policies
│   │   │       ├── jobs.py   # jobs + dead-letter-queue
│   │   │       ├── workers.py # registration, heartbeats, shutdown
│   │   │       ├── dashboard.py # overview stats
│   │   │       ├── websocket.py # /ws realtime endpoint
│   │   │       └── health.py  # DB + Redis health check
│   │   ├── auth/
│   │   │   ├── security.py  # JWT issue/verify, bcrypt hashing, API key generation
│   │   │   └── dependencies.py # Principal resolution, RBAC, project-scope checks
│   │   ├── models/          # SQLAlchemy 2.0 ORM — 9 files, 15 tables
│   │   │   ├── base.py      # Declarative base, timestamp/UUID mixins
│   │   │   ├── organization.py # User, Organization, OrganizationMember, Project
│   │   │   ├── queue.py      # Queue, RetryPolicy
│   │   │   ├── job.py        # Job, ScheduledJob
│   │   │   ├── worker.py     # Worker, WorkerHeartbeat
│   │   │   ├── execution.py  # JobExecution, JobLog
│   │   │   ├── dead_letter.py # DeadLetterEntry
│   │   │   ├── api_key.py    # APIKey
│   │   │   └── audit.py      # AuditLog
│   │   ├── schemas/         # Pydantic request/response models (mirrors models/)
│   │   ├── repositories/    # Data-access layer — one repo per model, Repository pattern
│   │   ├── services/        # Business logic — auth, project, queue, job, worker, dashboard, audit
│   │   ├── scheduler/
│   │   │   └── main.py       # Scheduler process entrypoint
```

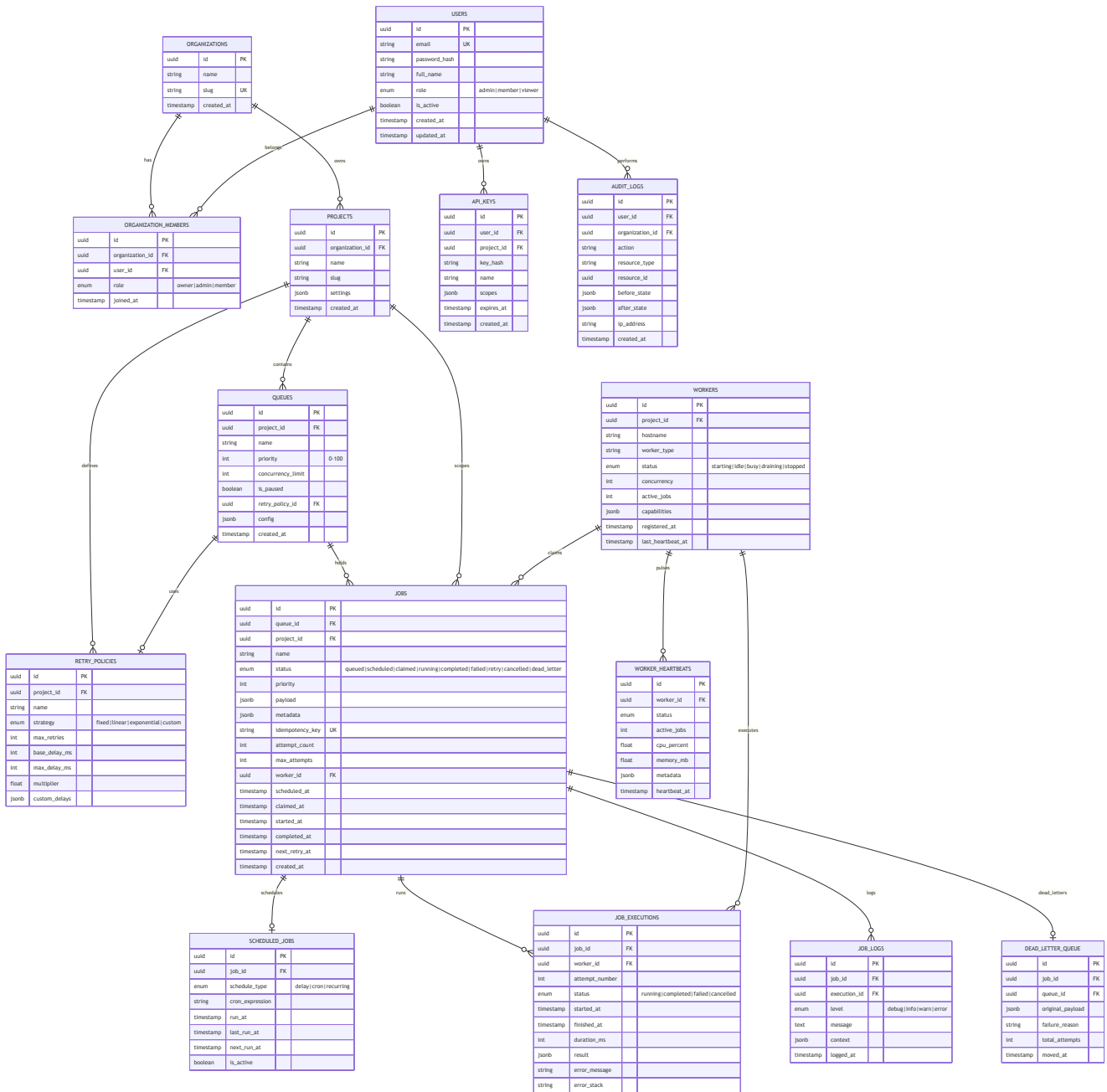
```

├── service.py          # Delay scanner + cron scanner
├── leader.py           # Redis-lock leader election for HA
├── workers/
│   ├── main.py         # Worker process entrypoint (signal handling)
│   └── worker.py        # Poll → claim → execute → heartbeat → graceful shutdown
├── execution/
│   ├── engine.py        # Runs one claimed job; decides retry vs. dead-letter
│   └── registry.py      # Pluggable task handlers (echo, sleep, http_request, ...)
├── retry/
│   └── policy.py        # fixed/linear/exponential/custom backoff math
├── queues/
│   ├── redis_client.py
│   ├── streams.py       # Redis Streams – claim wake-up signal
│   └── pubsub.py        # Redis Pub/Sub – WebSocket event bus
├── websocket/
│   └── hub.py           # Connection registry + broadcast
├── middleware/
│   ├── rate_limit.py    # Fixed-window, Redis-backed, per-IP
│   ├── request_logging.py # Structured request logs + Prometheus timing
│   └── exception_handler.py # Uniform {"error": ...} responses
├── database/
│   └── session.py       # Async engine + session factory
├── logging/
│   └── setup.py         # structlog JSON configuration
├── monitoring/
│   └── metrics.py       # Prometheus counters/histograms
├── tests/
│   ├── conftest.py      # Test DB/session/client fixtures
│   ├── unit/
│   │   ├── test_retry_policy.py
│   │   └── test_auth_security.py
│   └── integration/
│       ├── test_claim_engine.py    # Proves SKIP LOCKED never double-claims
│       ├── test_api_job_lifecycle.py
│       ├── test_validation_and_audit.py
│       └── test_schema_defaults.py # Regression guard for the now() default bug
├── frontend/
│   ├── package.json
│   ├── vite.config.ts
│   ├── index.html
│   ├── Dockerfile
│   ├── nginx.conf
│   └── src/
│       ├── main.tsx
│       ├── App.tsx          # Routes + provider tree
│       ├── index.css        # Neobrutalism design tokens (theme-aware)
│       ├── lib/
│       │   ├── api.ts       # axios instance + refresh-token interceptor
│       │   ├── types.ts     # Shared TS types mirroring backend schemas
│       │   ├── errors.ts    # Axios error → user-facing message
│       │   └── format.ts    # Relative time, duration, number formatting
│       ├── hooks/
│       │   ├── useRealtime.ts # WebSocket → React Query cache invalidation
│       │   ├── useTheme.ts   # Dark/light toggle, persisted
│       │   └── useDebounce.ts
│       ├── context/
│       │   ├── AuthContext.tsx
│       │   ├── ProjectContext.tsx # Current org/project selection
│       │   └── ToastContext.tsx
│       ├── components/
│       │   ├── Layout.tsx    # Sidebar nav + realtime indicator
│       │   ├── ProtectedRoute.tsx
│       │   └── ui/           # Button, Card, Modal, Badge, ConfirmDialog, Skeleton, Input
│       └── pages/

```

- └ Login.tsx / Register.tsx
- └ Dashboard.tsx # Overview cards, charts, onboarding checklist
- └ Queues.tsx # Priority, concurrency, pause/resume, retry policies
- └ Jobs.tsx / JobDetail.tsx
- └ Workers.tsx # Live status + copy-paste start command
- └ DeadLetterQueue.tsx
- └ ApiKeys.tsx
- └ AuditLog.tsx

4. Database ER Diagram

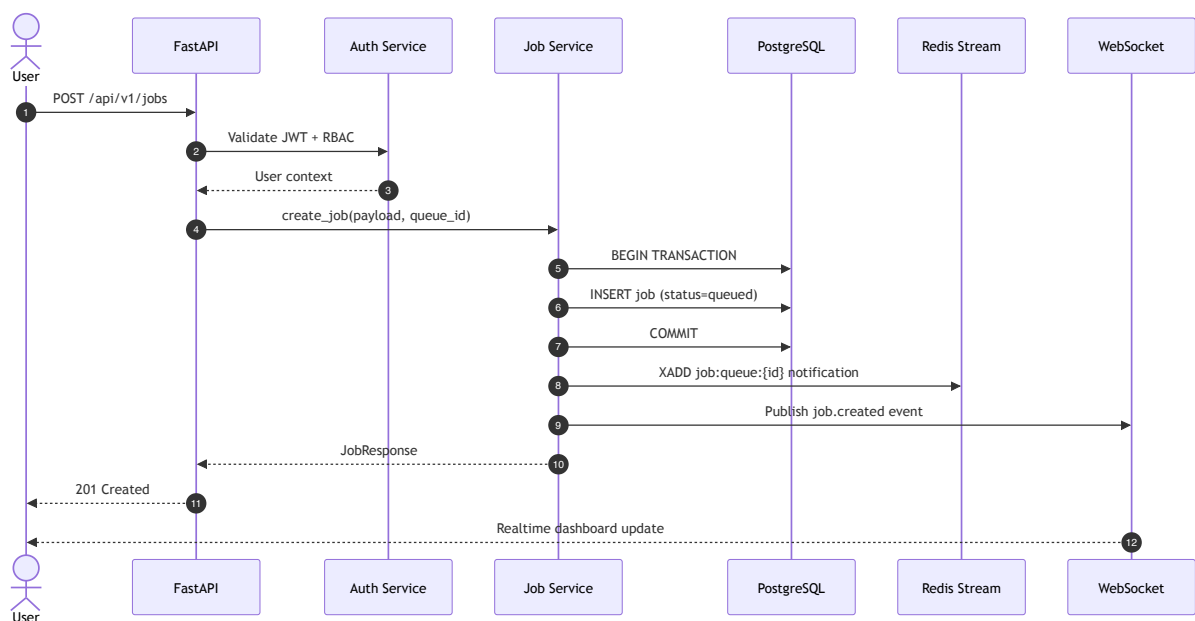


Key Indexes

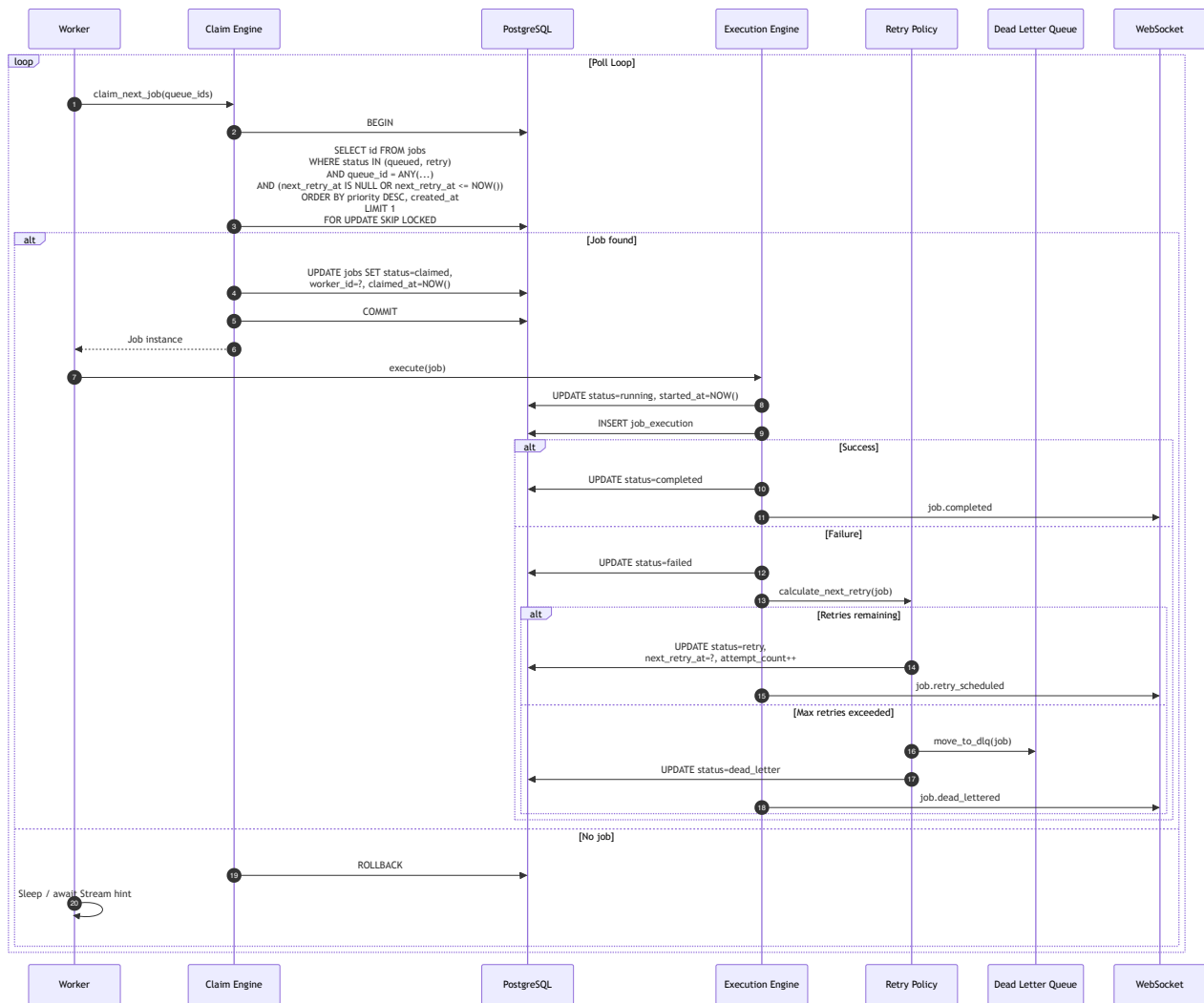
Table	Index	Purpose
jobs	(queue_id, status, priority DESC, created_at)	Priority claim query
jobs	(status, scheduled_at) WHERE status = 'scheduled'	Delay scanner
jobs	(status, next_retry_at) WHERE status = 'retry'	Retry scanner
jobs	(idempotency_key) UNIQUE	Dedup
jobs	(project_id, status, created_at DESC)	Dashboard listing
workers	(project_id, status, last_heartbeat_at)	Health checks
job_executions	(job_id, attempt_number)	History lookup
audit_logs	(organization_id, created_at DESC)	Compliance queries

5. Sequence Diagrams

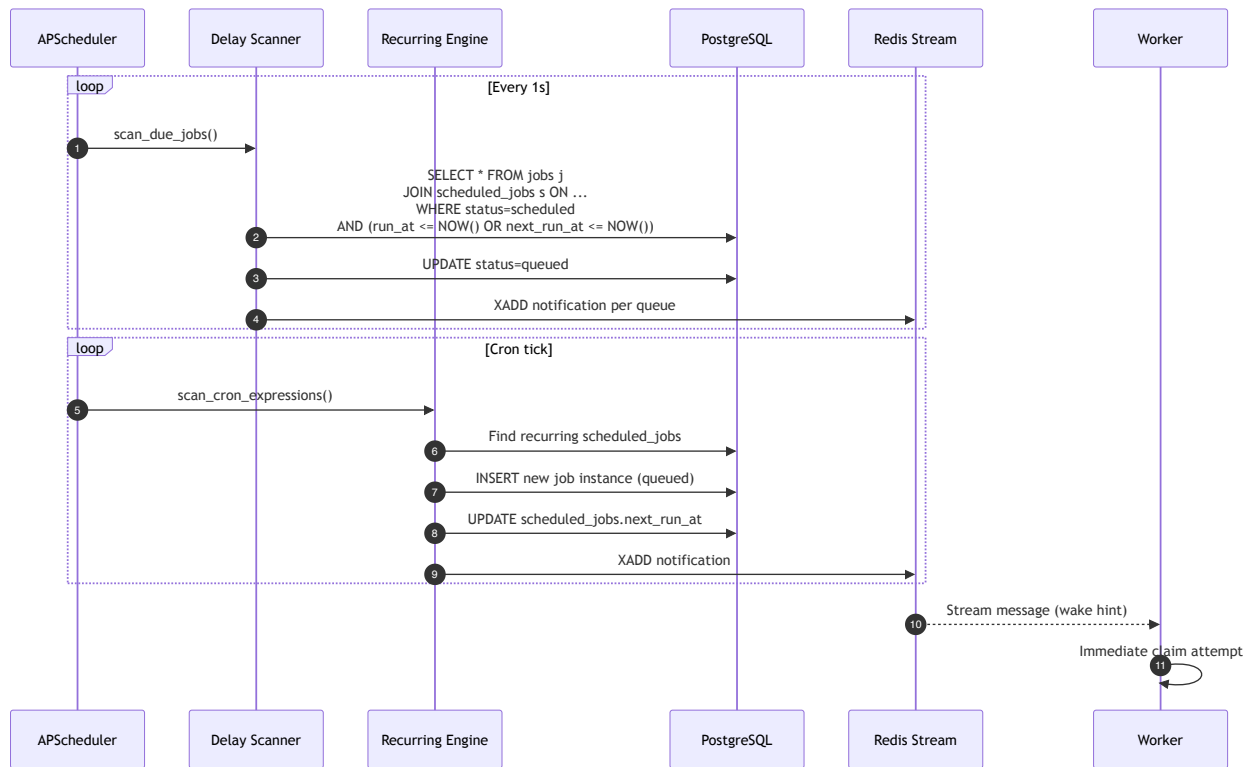
5.1 Job Creation (Immediate)



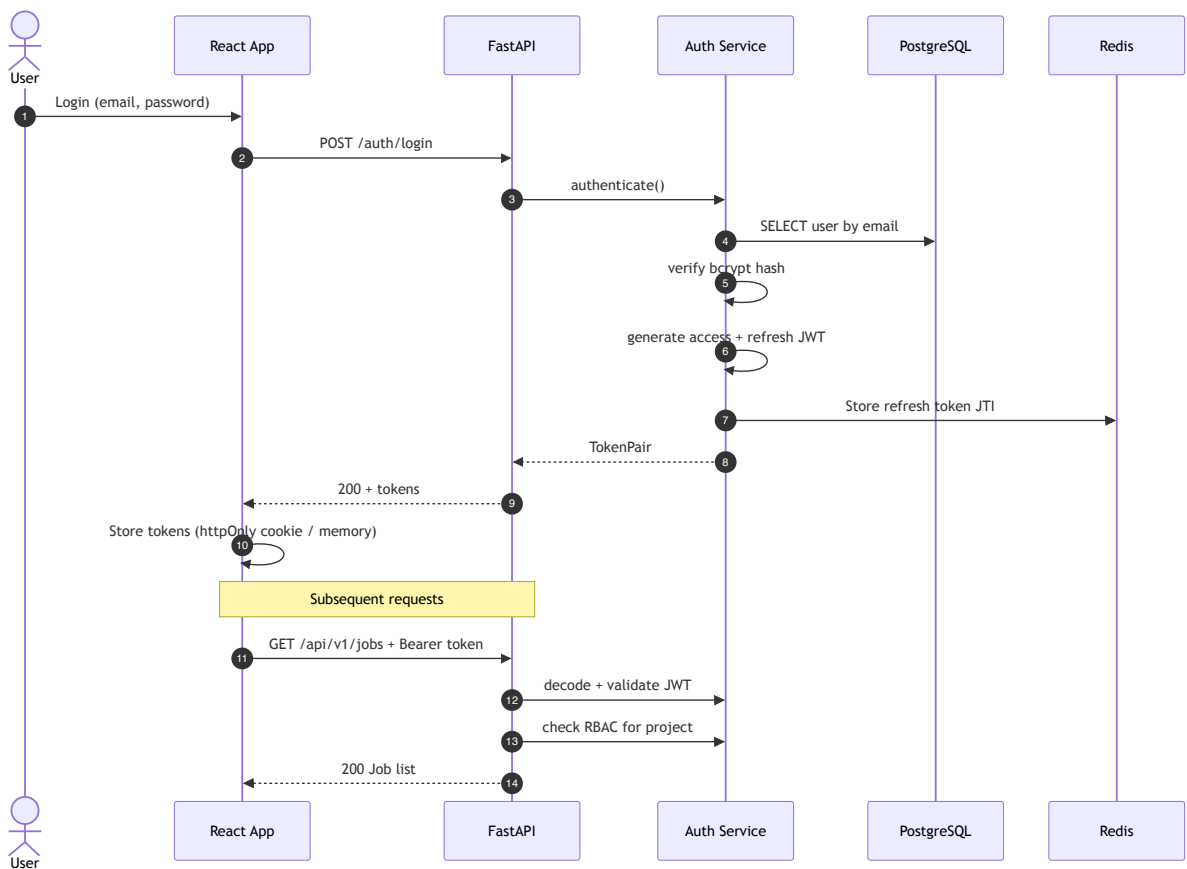
5.2 Atomic Job Claim & Execution



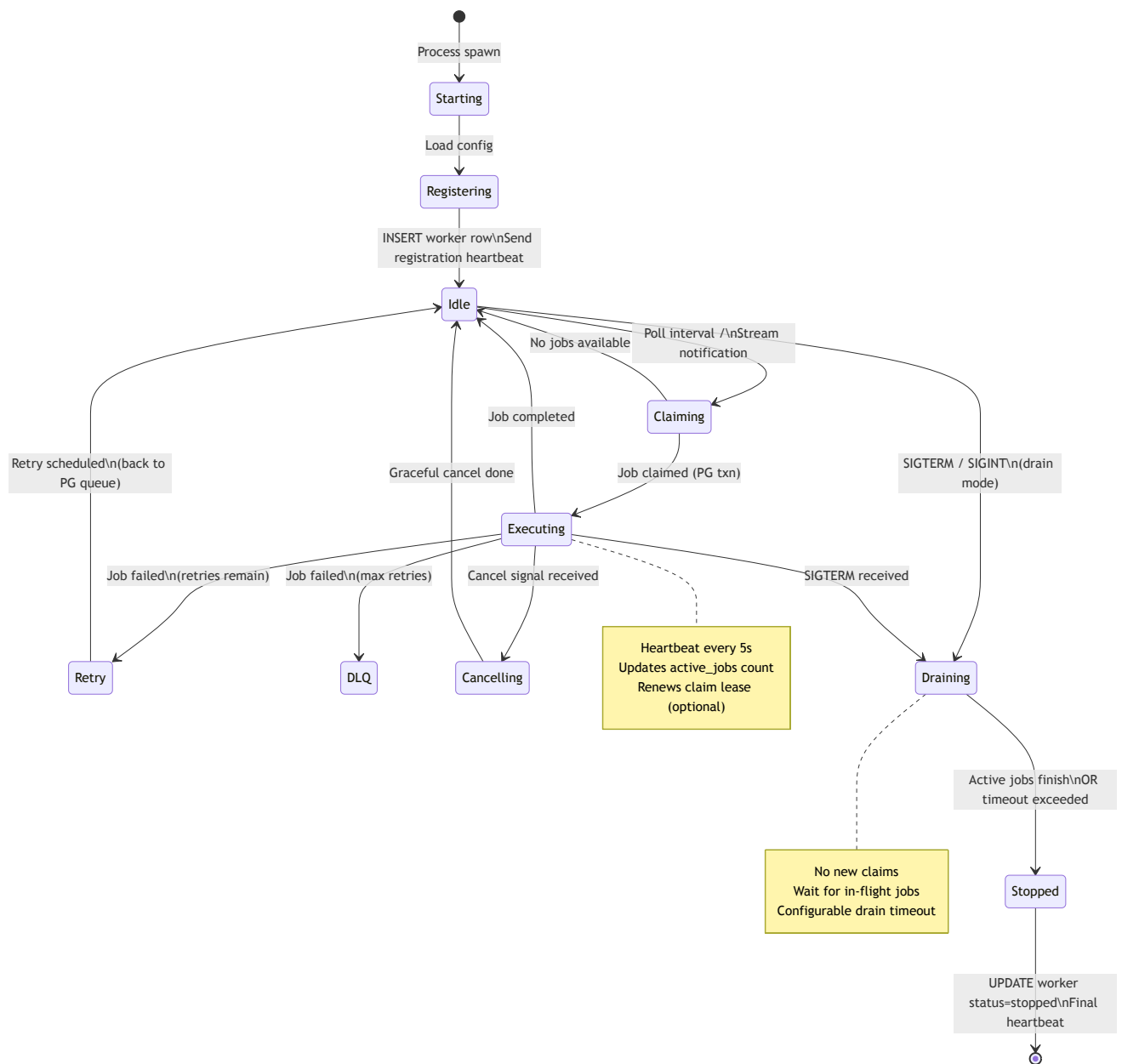
5.3 Scheduled / Cron Job Activation



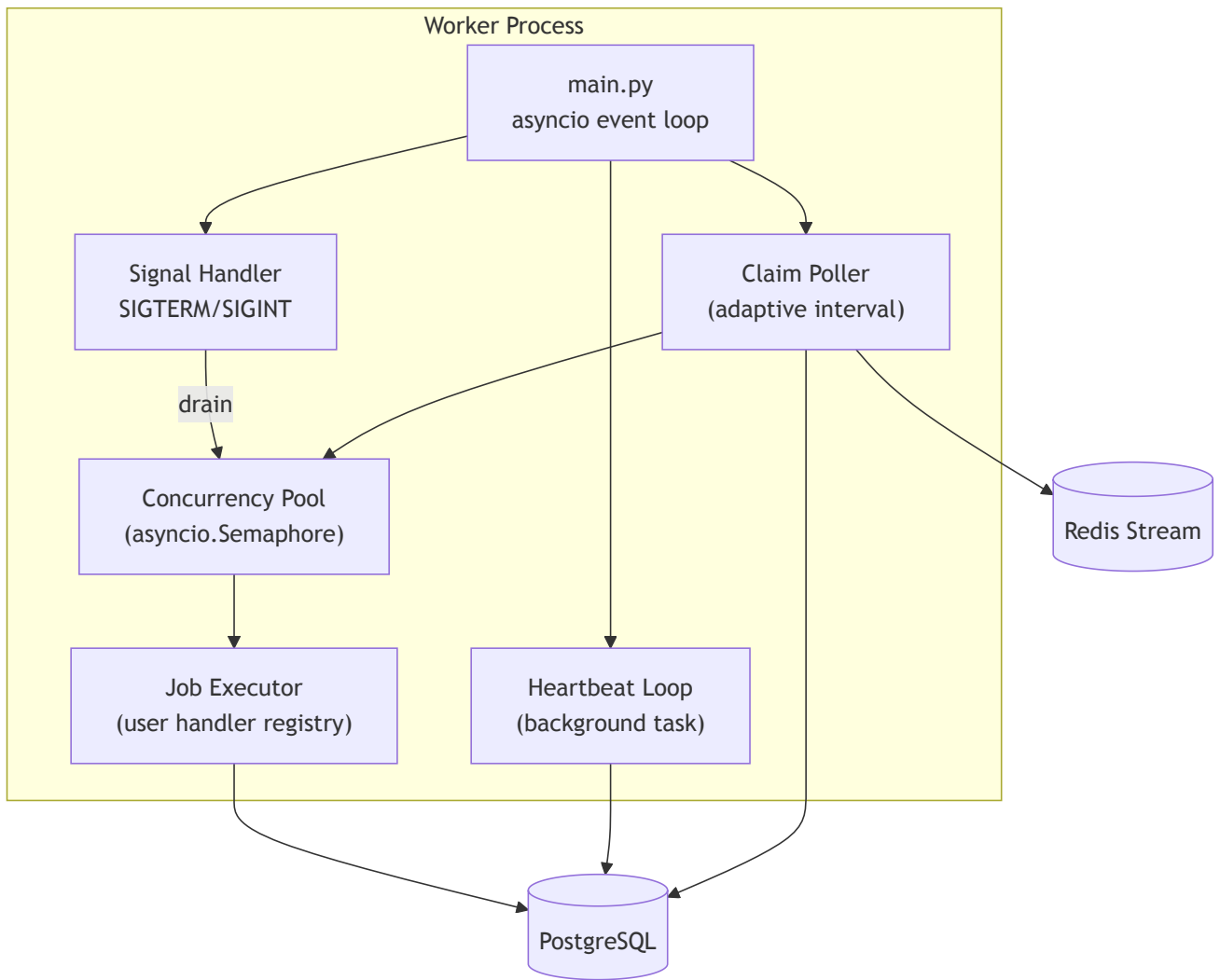
5.4 Authentication Flow



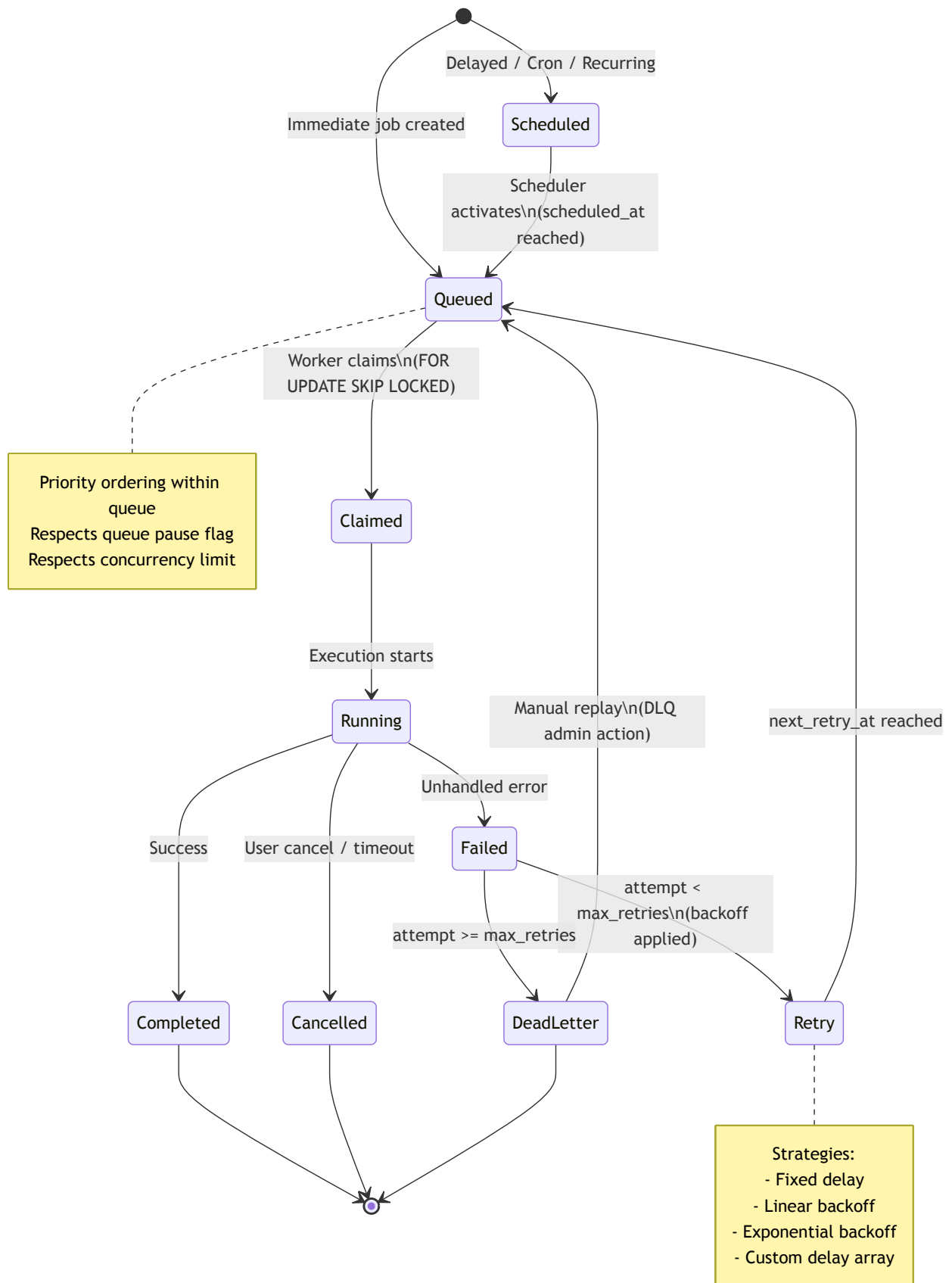
6. Worker Lifecycle Diagram



Worker Internal Architecture



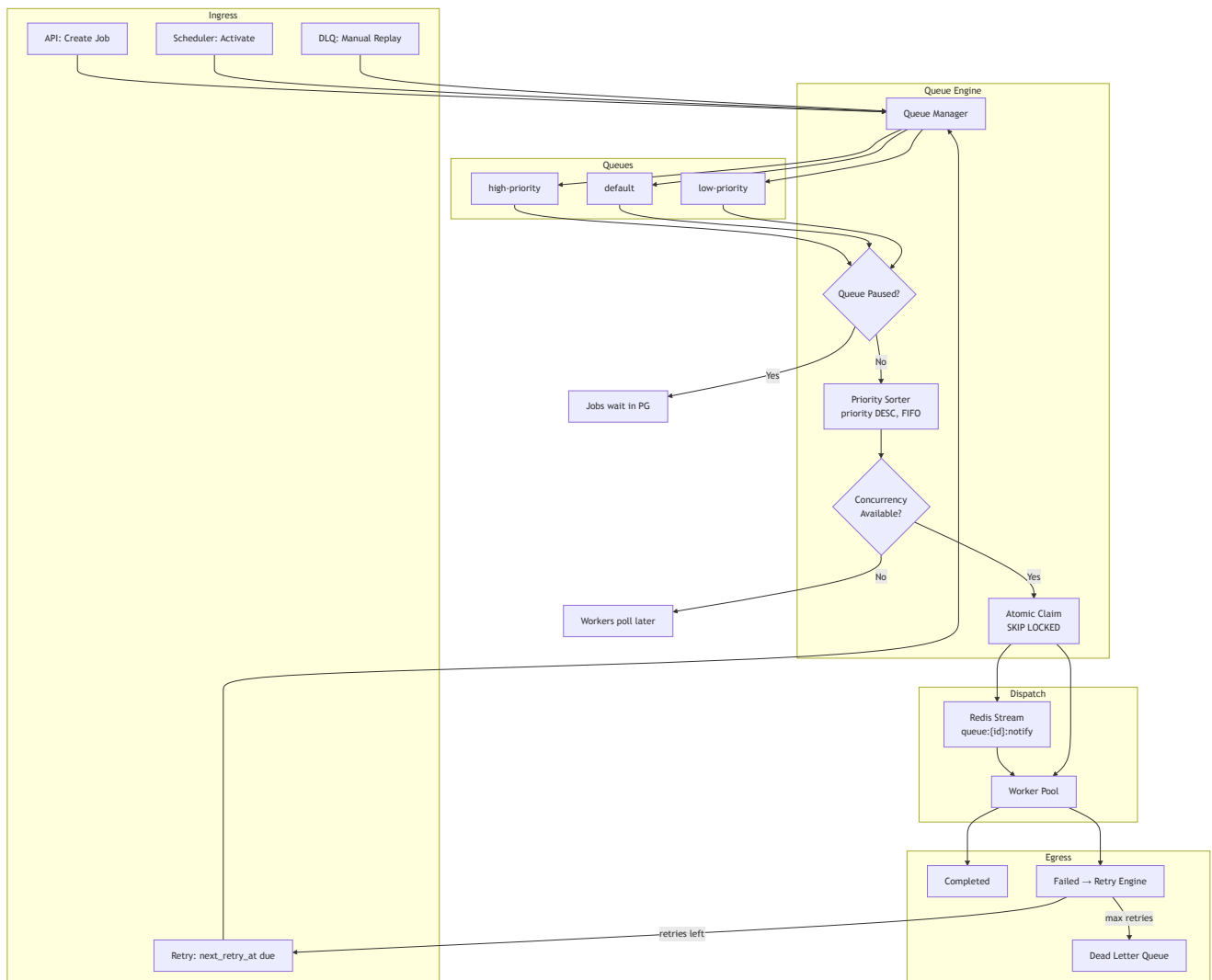
7. Job Lifecycle Diagram



Job Type Matrix

Type	Initial State	Trigger	Re-queue
Immediate	queued	API create	No
Delayed	scheduled	scheduled_at	No
Cron	scheduled	Cron expression	Yes — new instance each tick
Recurring	scheduled	Interval / cron	Yes
Batch	queued (parent)	Parent dispatches children	Children independent

8. Queue Flow Diagram



Priority Queue Implementation

Jobs are **not** stored in Redis priority queues (which lack durability). PostgreSQL holds ordering:

```

-- Claim query (simplified)
SELECT j.id
FROM jobs j
JOIN queues q ON j.queue_id = q.id
WHERE j.status IN ('queued', 'retry')

```

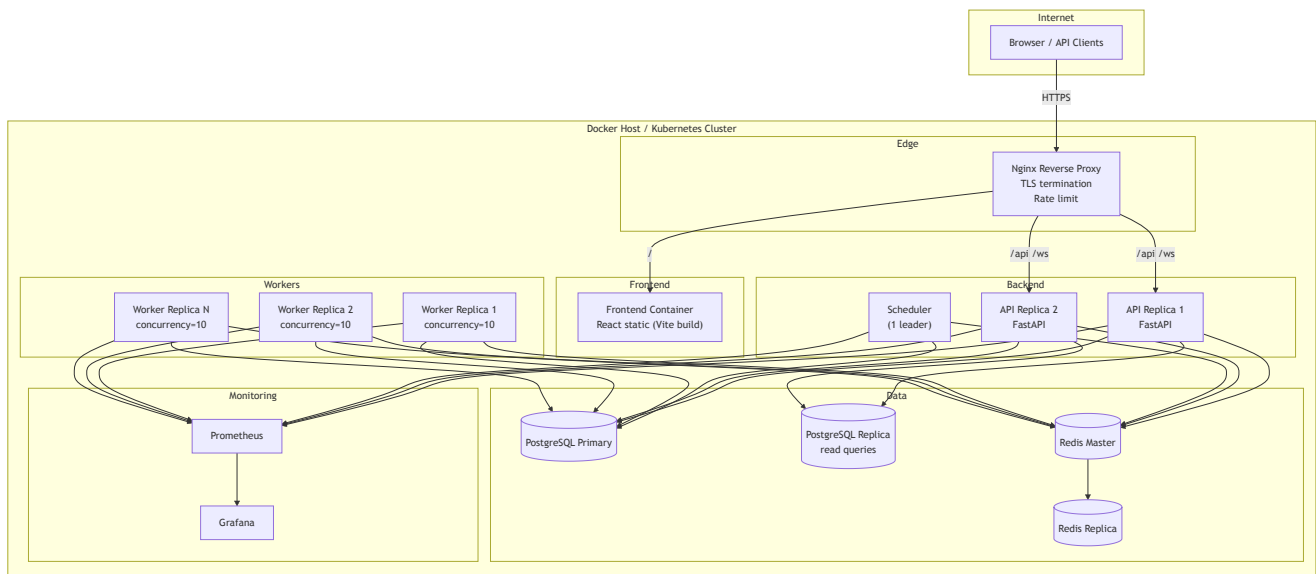
```

AND q.is_paused = false
AND j.queue_id = ANY(:worker_queue_ids)
AND (j.next_retry_at IS NULL OR j.next_retry_at <= NOW())
ORDER BY j.priority DESC, j.created_at ASC
LIMIT 1
FOR UPDATE OF j SKIP LOCKED;

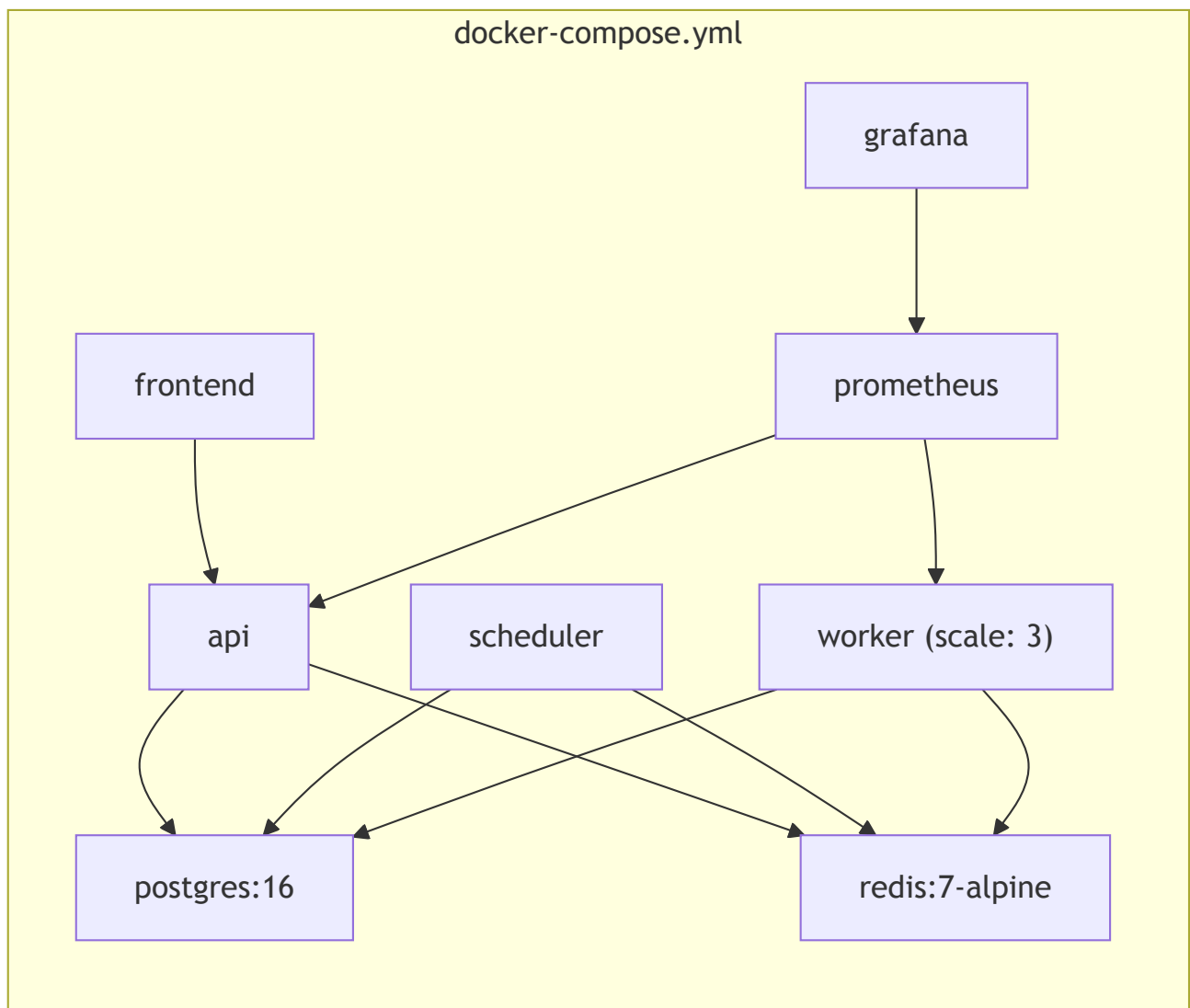
```

Queue-level `concurrency_limit` is enforced by counting active (`claimed` , `running`) jobs per queue before claim.

9. Deployment Diagram



Docker Compose Services (Development)



Environment Profiles

Profile	API	Scheduler	Workers	Postgres	Redis
dev	1	1	1	1	1
staging	2	1 (+ standby)	3	1 + replica	Sentinel
production	3+	Leader election	Auto-scale	HA cluster	Cluster

10. Component Rationale

Why each component exists

Component	Why It Exists
React Dashboard	Operators need visibility into queues, workers, failures, and throughput without querying SQL. Neobrutalism theme provides high-contrast, scannable enterprise UI.
FastAPI API	Typed, async, auto-documented REST + WebSocket hub. Control plane for all job/queue/worker operations.

Component	Why It Exists
PostgreSQL	ACID transactions for atomic job claiming. Relational model fits orgs → projects → queues → jobs hierarchy. <code>SKIP LOCKED</code> is the industry-standard pattern (used by Graphile Worker, River, etc.).
Redis Streams	Low-latency wake-up notifications so workers don't hammer DB on idle polls. Stream consumer groups allow partitioned dispatch.
Redis Cache	JWT blocklist, rate limit counters, dashboard metric snapshots, distributed lock for scheduler leader.
Redis Pub/Sub	Fan-out realtime events to WebSocket connections across API replicas.
APScheduler	Battle-tested cron/interval scanning. Runs in dedicated scheduler process to keep API latency isolated.
Scheduler Service	Separates time-based activation from request handling. Scans <code>scheduled</code> → <code>queued</code> transitions without blocking API.
Worker Processes	Horizontally scalable execution plane. Crash of one worker doesn't affect others; unclaimed jobs remain in PG.
Claim Engine	Encapsulates <code>FOR UPDATE SKIP LOCKED</code> logic — the core correctness primitive preventing duplicate execution.
Execution Engine	Runs job handlers with concurrency control, cancellation, timeout, and result persistence.
Retry Engine	Centralizes backoff math (fixed/linear/exponential/custom). Keeps worker code simple.
Dead Letter Queue	Failed jobs need a quarantine area for inspection, alerting, and manual replay — not silent loss.
Heartbeat System	Detects stale workers. Enables orchestrator to mark crashed workers and optionally release stale claims.
WebSocket Hub	Dashboard expects live updates (job state changes, worker status) without polling.
Repository Pattern	Decouples SQL from business logic. Enables unit testing services with mock repos.
Service Layer	Orchestrates transactions, validation, and cross-entity rules (e.g., can't enqueue to paused queue).
Prometheus Metrics	Industry standard for alerting on queue depth, failure rate, worker count, claim latency.
Structured JSON Logs	Machine-parseable logs for ELK/Datadog. Correlation IDs tie API request → job → execution.
JWT + Refresh Tokens	Stateless auth for API with revocable refresh via Redis.
RBAC	Multi-tenant org/project isolation. Roles: owner, admin, member, viewer.
API Keys	Machine-to-machine job submission without user sessions.
Audit Logs	Compliance trail for who changed queue config, replayed DLQ jobs, etc.
Alembic Migrations	Version-controlled schema evolution for production deployments.
Nginx	TLS, routing, rate limiting at edge. Serves static frontend separately from API.

Bonus Features — Architecture Hooks

Feature	Approach
Workflow DAG	<code>jobs.metadata.parent_job_id</code> + dependency table; child jobs start when parent completes
Queue Sharding	<code>queues.config.shard_key</code> — workers subscribe to shard subsets
Distributed Locking	Redis Redlock for scheduler leader election
Leader Election	Scheduler standby acquires lock; only leader runs cron scans
AI Failure Summary	Post-execution hook sends error stack to LLM endpoint; stores summary in <code>job_executions.result</code>
Execution Replay	DLQ → clone job with new idempotency key → re-enqueue
Priority Inheritance	Child jobs inherit parent priority unless overridden
Rate Limiting	Redis sliding window per org/project on API; per-queue token bucket for job creation

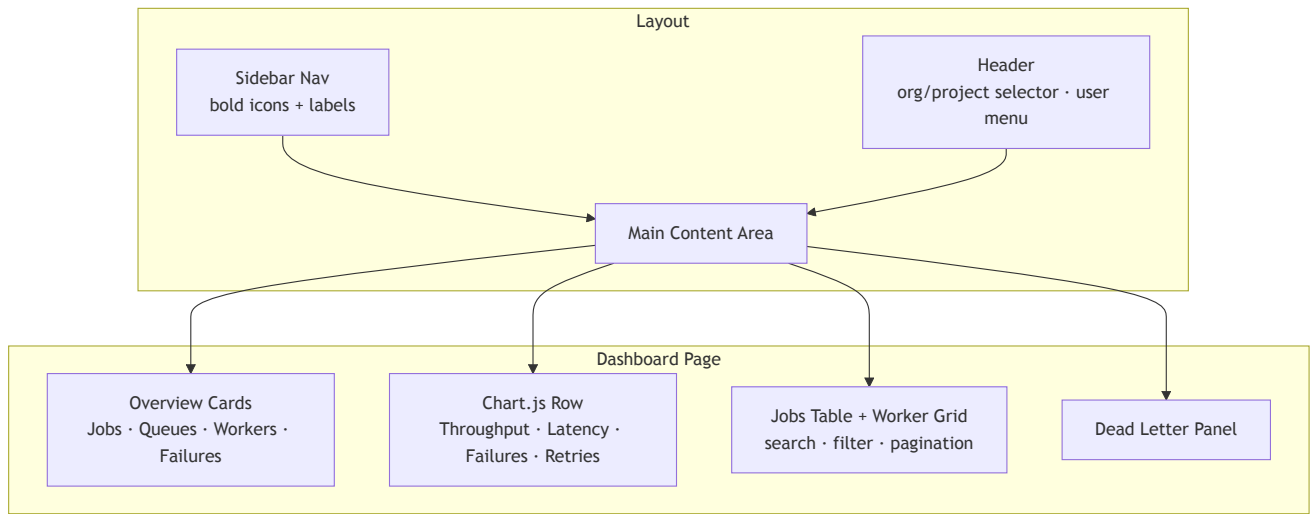
11. Frontend Design — Neobrutalism Theme

The dashboard uses **Neobrutalism** — bold borders, hard shadows, high contrast, flat colors — applied on top of ShadCN UI primitives.

Design Tokens

```
/* Core neobrutalism tokens */
--neo-border: 3px solid #000;
--neo-shadow: 4px 4px 0px #000;
--neo-shadow-hover: 6px 6px 0px #000;
--neo-radius: 0px;          /* Sharp corners */
--neo-bg: #FFFDF5;          /* Warm off-white */
--neo-primary: #FF6B35;     /* Orange accent */
--neo-secondary: #004E89;   /* Deep blue */
--neo-success: #06D6A0;
--neo-warning: #FFD166;
--neo-danger: #EF476F;
--neo-muted: #F0EDE5;
--neo-font: 'Space Grotesk', sans-serif;
```

Dashboard Layout



Key UI Components

Component	Purpose
NeoCard	Wrapper with 3px border + offset shadow
NeoButton	ShadCN Button with brutal shadow + press animation
OverviewCards	4-up grid: total jobs, active workers, queue depth, failure rate
ThroughputChart	Jobs/min line chart (Chart.js)
JobsTable	Sortable, filterable, paginated with status badges
WorkerGrid	Live heartbeat indicators (green/yellow/red)
RealtimeProvider	WebSocket context feeding React Query cache invalidation

12. Technology Decisions Summary

Decision	Choice	Alternatives Considered	Rationale
Job store	PostgreSQL	Redis-only, DynamoDB	ACID claiming, complex queries, audit
Wake-up signal	Redis Streams	Polling only, RabbitMQ	Already in stack; persistent consumer groups
Claim pattern	SKIP LOCKED	Advisory locks, Redis BRPOPLPUSH	PostgreSQL-native, no extra broker dependency for correctness
Scheduler	APScheduler	Celery Beat, custom cron	Lightweight, embeddable, sufficient for scan workloads
API framework	FastAPI	Django, Flask	Async, Pydantic, OpenAPI auto-gen
Frontend	React + Vite	Next.js	SPA dashboard; no SSR needed
Auth	JWT + Refresh	Session cookies only	Stateless API replicas; refresh revocable in Redis
Metrics	Prometheus	Datadog agent	Open-source, pull model, Grafana ecosystem
Migrations	Alembic	Raw SQL	SQLAlchemy 2.0 integration

Next Steps

Once this architecture is **approved**, implementation proceeds in phases:

Phase	Scope	Deliverable
1	Database schema + migrations + core models	Alembic migrations, SQLAlchemy models
2	Auth + Organizations + Projects + Queues API	JWT auth, CRUD endpoints
3	Job engine + claim + state machine + retry	Worker process, claim engine, DLQ
4	Scheduler (delay, cron, recurring)	APScheduler service

Phase	Scope	Deliverable
5	WebSocket + Redis Pub/Sub	Live dashboard feed
6	Frontend dashboard (neobrutalism)	Full React UI
7	Observability + Docker Compose	Prometheus, health checks, <code>docker compose up</code>
8	Tests + documentation	Pytest suite, API.md, Deployment.md

Please review this architecture and confirm approval (or note changes) before implementation begins.

Database & ER Diagram

[View on GitHub: docs/Database.md](#)

Database

PostgreSQL is the single source of truth for all durable state. Redis is a cache/notification layer only — nothing there is required for correctness, and losing it degrades latency, not data.

Schema is managed by Alembic (`backend/migrations/`); the mapped models live in `backend/app/models/` . See [Architecture.md](#) for the full ER diagram.

Tables

Table	Purpose
<code>users</code>	Login identity, global role, bcrypt password hash
<code>organizations</code>	Top-level tenant boundary
<code>organization_members</code>	User ↔ organization, with an org-scoped role (<code>owner</code> / <code>admin</code> / <code>member</code>)
<code>projects</code>	Groups queues/jobs/workers under one organization
<code>retry_policies</code>	Named backoff configs (<code>fixed</code> / <code>linear</code> / <code>exponential</code> / <code>custom</code>), reusable across queues
<code>queues</code>	Priority, concurrency limit, pause flag, optional retry policy
<code>jobs</code>	One row per job instance — status, payload, attempt count, scheduling fields
<code>scheduled_jobs</code>	1:1 with a job; drives delayed/cron/recurring firing (see below)
<code>workers</code>	Registered worker processes, heartbeat timestamp, concurrency
<code>worker_heartbeats</code>	Time-series of heartbeat samples (cpu/memory/active_jobs) per worker
<code>job_executions</code>	One row per attempt — start/finish, duration, result or error
<code>job_logs</code>	Structured log lines attached to a job/execution
<code>dead_letter_queue</code>	Jobs that exhausted <code>max_attempts</code> , with the original payload preserved
<code>api_keys</code>	Hashed API keys (SHA-256), optionally scoped to one project
<code>audit_logs</code>	Who did what, before/after state, for compliance

Why Postgres, not just Redis

Redis Streams give you a fast queue, but Sidekiq/BullMQ/RabbitMQ-Redis backends all suffer the same failure mode: if Redis loses data (eviction, restart without AOF, failover), jobs vanish silently. Postgres gives ACID guarantees, foreign keys that make orphaned state impossible, and — critically — `SELECT ... FOR UPDATE SKIP LOCKED`, which is what makes atomic claiming possible without a separate distributed lock service.

The claim query

This is the mechanism that lets N workers poll the same queue without ever double-executing a job:

```
BEGIN;

SELECT id FROM jobs
WHERE queue_id = :queue_id
  AND status IN ('queued', 'retry')
  AND (scheduled_at IS NULL OR scheduled_at <= now())
  AND (next_retry_at IS NULL OR next_retry_at <= now())
ORDER BY priority DESC, created_at ASC
LIMIT :limit
FOR UPDATE SKIP LOCKED;

UPDATE jobs SET status = 'claimed', claimed_at = now(), worker_id = :worker_id
WHERE id IN (<ids from above>)
RETURNING *;

COMMIT;
```

`SKIP LOCKED` means a second worker's identical query — running concurrently — simply skips any row already locked by the first worker's open transaction, instead of blocking on it. No two workers ever see the same row as a candidate. This is implemented in `app/repositories/job.py`'s `claim_jobs()`, and is covered by a concurrency test

(`tests/integration/test_claim_engine.py::test_concurrent_claims_never_double_claim_the_same_job`) that fires 4 simulated workers at 20 jobs simultaneously and asserts zero duplicates.

Indexes worth knowing about

- `ix_jobs_claim_candidates (queue_id, status, priority, scheduled_at)` — covers the claim query's `WHERE` + `ORDER BY` so it doesn't degrade to a sequential scan as the `jobs` table grows.
- `ix_jobs_project_status (project_id, status)` — backs the dashboard's status-breakdown counts and the jobs list's status filter.
- `uq_project_idempotency_key (project_id, idempotency_key)` — enforces idempotent job creation.

Cascade rules

- Deleting an `organization` cascades to its `projects`, which cascades to `queues`, `retry_policies`, and (via `queues`) `jobs`.
- Deleting a `job` cascades to its `job_executions`, `job_logs`, `scheduled_jobs` row, and `dead_letter_queue` entry.
- Deleting a `worker` sets `jobs.worker_id` and `job_executions.worker_id` to `NULL` (history is kept; only the live pointer is cleared) and cascades its `worker_heartbeats`.

Recurring / cron jobs

There's no separate "job template" table. A cron/recurring job is a normal `jobs` row that stays in `scheduled` status forever, paired with a `scheduled_jobs` row (`schedule_type = 'cron'`) holding the cron expression and `next_run_at` . Every scheduler tick, `app/scheduler/service.py::scan_cron_jobs` finds due schedules, **clones** the template job into a fresh `queued` row (a real, independent job with its own execution history), and advances `next_run_at` via `croniter` . The template itself is never claimed or executed — only its clones are.

API Documentation

[View on GitHub: docs/API.md](#)

API

Full interactive reference (request/response schemas, try-it-out) is auto-generated by FastAPI and served at `/docs` (Swagger UI) and `/redoc` whenever the API is running, plus the raw schema at `/openapi.json` . This document is a map of what exists and how auth/pagination work — treat `/docs` as the source of truth for exact payloads.

Base path: `/api/v1` . Realtime events: `GET /api/v1/ws` (WebSocket).

Authentication

Two schemes, both sent as `Authorization: Bearer <token>` :

1. **JWT access tokens** — from `POST /auth/register` or `POST /auth/login` . Short-lived (`access_token_expire_minutes` , default 30 min); refresh via `POST /auth/refresh` with the longer-lived `refresh_token` (default 7 days).
2. **API keys** — created with `POST /auth/api-keys` (shown once, prefixed `djs_`), optionally scoped to a single project. Pass the raw key as the bearer token; the API detects the `djs_` prefix and looks it up by SHA-256 hash.

Endpoints under `/projects/{project_id}/...` additionally check organization membership (JWT) or project scope (API key) via the `require_project_access` dependency.

Endpoints by resource

Resource	Endpoints
Auth	<code>POST /auth/register</code> , <code>POST /auth/login</code> , <code>POST /auth/refresh</code> , <code>GET /auth/me</code> , <code>POST /auth/api-keys</code>
Organizations	<code>GET /organizations</code> , <code>GET/POST /organizations/{id}/projects</code> , <code>PATCH /organizations/{id}/projects/{project_id}</code>
Queues	<code>GET/POST /projects/{id}/queues</code> , <code>PATCH /projects/{id}/queues/{queue_id}</code> , <code>POST .../pause</code> , <code>POST .../resume</code> , <code>GET .../metrics</code>
Retry policies	<code>GET/POST /projects/{id}/retry-policies</code>
Jobs	<code>POST /projects/{id}/jobs</code> , <code>POST .../batch</code> , <code>GET /projects/{id}/jobs</code> (paginated/filtered/searched), <code>GET/POST .../{job_id}</code> , <code>.../cancel</code> , <code>.../replay</code> , <code>.../executions</code> , <code>.../logs</code>

Resource	Endpoints
Dead letter queue	<code>GET /projects/{id}/dead-letter-queue?queue_id=...</code> (paginated)
Workers	<code>POST/GET /projects/{id}/workers</code> , <code>POST .../{worker_id}/heartbeat</code> , <code>GET .../heartbeats</code> , <code>POST .../shutdown</code>
Dashboard	<code>GET /projects/{id}/dashboard/overview</code>
Ops	<code>GET /health</code> (DB + Redis check), <code>GET /metrics</code> (Prometheus exposition format)

Pagination, filtering, sorting

List endpoints that return many rows (`jobs` , `dead-letter-queue`) return a `Page<T>` envelope:

```
{ "items": [...], "total": 137, "page": 1, "page_size": 20, "pages": 7 }
```

Query params: `page` , `page_size` (max 200), and for jobs specifically `status` , `queue_id` , `search` (substring match on job name), `sort_by` , `sort_order` (`asc` / `desc`).

Creating a job

`POST /projects/{id}/jobs` — exactly one of `run_at` , `delay_seconds` , `cron_expression` may be set; omitting all three runs the job immediately.

```
{
  "name": "send-welcome-email",
  "queue_id": "...",
  "payload": { "task": "http_request", "args": { "url": "https://..." } },
  "priority": 10,
  "max_attempts": 5,
  "delay_seconds": 30
}
```

`payload.task` selects a handler from the built-in registry (`app/execution/registry.py`): `echo` , `sleep` , `http_request` , `fail` , `random_fail` . Registering a new task is a one-line `@task("name")` -decorated async function — see that file for the pattern.

Errors

Non-2xx responses are `{"error": "message"}` (validation failures additionally include a `details` array of Pydantic error objects). HTTP status codes follow convention: 401 (missing/invalid credentials), 403 (authenticated but not authorized for this org/project/role), 404, 409 (conflicts — duplicate queue name, terminal-state job cancellation), 422 (validation), 429 (rate limit).

Design Decisions

[View on GitHub: docs/Design-Decisions.md](#)

Design Decisions

Short explanations of the choices that shape this system, and what they trade off against.

Postgres as the queue, Redis as a doorbell

BullMQ and most Redis-based queues make Redis the source of truth. That's fast, but it means a Redis data-loss event (eviction under memory pressure, a restart without AOF fsync, a botched failover) silently drops jobs. This system instead keeps every durable fact — job state, attempt count, execution history — in Postgres, and uses Redis purely as a low-latency wake-up signal (Streams) and fan-out bus (Pub/Sub for the WebSocket hub). If Redis disappears entirely, workers fall back to polling Postgres directly on `WORKER_POLL_INTERVAL_MS`; nothing is lost, throughput just degrades to poll-interval latency instead of near-instant.

`SELECT ... FOR UPDATE SKIP LOCKED` instead of a distributed lock

The alternative designs — a Redis `SETNX` lock per job, or a dedicated coordinator service — both add a second system that has to agree with Postgres about job state, which is exactly the kind of two-phase-commit problem you don't want. `SKIP LOCKED` gets the same atomicity guarantee for free from a database transaction: two workers running the identical claim query concurrently will never see the same row, because the second worker's query simply skips whatever the first worker's open transaction has locked. No lock service, no lease renewal, no split-brain window. The cost is that this only works because Postgres is already the source of truth — it wouldn't help if job state lived in Redis.

`job.max_attempts` as the retry cap, not `retry_policy.max_retries`

Both fields exist on purpose but answer different questions: `RetryPolicy` (attached to a *queue*) answers "how should the delay between attempts grow" — fixed, linear, exponential, or a custom list. `Job.max_attempts` (set per job at creation time) answers "how many times should *this* job be tried." Early in implementation the execution engine used the policy's `max_retries` for both, which meant a job created with `max_attempts: 2` actually got 4 attempts (the queue's default policy allowed 3 retries beyond the first try) — caught by manual end-to-end testing, not by a unit test, because the unit tests for each piece were individually correct. The fix: the engine reads `job.max_attempts` to decide whether to retry or dead-letter, and asks the retry policy only for the delay. Lesson generalized: when two config sources can each independently answer "how many," decide upfront which one wins.

`server_default=func.now()` , never `server_default="now()"`

A second bug caught by end-to-end testing rather than unit tests, and worth documenting because it's easy to reintroduce: nine columns across eight models (`jobs.created_at`, `queues.created_at`, `api_keys.created_at`, `audit_logs.created_at`, `organization_members.joined_at`, `workers.registered_at`, `worker_heartbeats.heartbeat_at`, `job_logs.logged_at`, `dead_letter_queue.moved_at`) were declared with `server_default="now()"` — a bare Python string. SQLAlchemy compiles a bare string server default as a **quoted SQL string literal**, not a function call. Postgres then parses that literal once, at the moment the `CREATE TABLE` / `ALTER TABLE` DDL runs, and freezes it into a fixed timestamp default — so *every row ever inserted afterward* got the exact same `created_at`, down to the microsecond, no matter when it was actually created. Unit tests never caught it because each test run creates its own schema fresh and finishes in milliseconds, so "every row has an identical timestamp" looked indistinguishable from "every row was created quickly." It surfaced as relative-time UI (job lists, audit log) showing implausible ages like "2h ago" for a job created seconds earlier.

The fix is `server_default=func.now()` (a live SQL function reference, compiles to `DEFAULT now()` with no quotes), which Postgres evaluates fresh per row. Two follow-ups to make sure this specific mistake can't silently ship again: `migrations/env.py` now sets `compare_server_default=True`, because Alembic's autogenerate

ignores `server_default` changes entirely unless you opt in — the initial migration was generated *before* that flag existed, which is exactly how nine wrong defaults made it into a committed migration without a diff ever flagging them. And `tests/integration/test_schema_defaults.py` asserts every timestamp-default column's `information_schema.columns.column_default` contains `now()` and isn't a quoted literal.

Cron/recurring jobs clone instances rather than templating at execution time

The ER schema ties `scheduled_jobs` to a job 1:1. Rather than inventing a separate "recurring job definition" table, a cron job is a normal job that never leaves `scheduled` status — the scheduler's cron scanner clones it into a fresh, independently-tracked `queued` job on every firing and advances the template's `next_run_at`. This keeps the schema uniform (every job has the same execution/log/DLQ relationships, whether it came from a one-off create or a cron firing) at the cost of one extra row per firing, which is a fine trade for the audit trail it buys you — every recurring firing has its own full execution history instead of overwriting a single row.

Workers are scoped to one project

`workers.project_id` is `NOT NULL`. This mirrors how BullMQ/Sidekiq workers are typically deployed — one worker fleet per logical service/queue namespace — rather than a single global worker pool serving every tenant. It also keeps the claim query cheap (index on `queue_id`, not a cross-tenant scan) and means a noisy tenant can't starve another tenant's concurrency slots. The trade-off: multi-project deployments run one worker process (or fleet) per project rather than sharing one pool, which is more processes to operate but gives hard isolation for free.

Leader election for the scheduler, not for workers

Workers don't need leader election — `SKIP LOCKED` already makes concurrent claiming safe, so you scale workers by just running more of them. The scheduler is different: `scan_delayed_jobs` and `scan_cron_jobs` aren't idempotent-by-locking the way claiming is (nothing stops two scheduler replicas from both flipping the same delayed job to `queued`, which is harmless, or both cloning the same cron firing, which duplicates work). A cheap Redis `SET NX EX` lock, renewed on an interval well inside its TTL, lets you run multiple scheduler replicas for availability while guaranteeing only one is actively scanning at a time — if the leader dies, the lock expires and another replica picks it up within `SCHEDULER_LEADER_LOCK_TTL_SEC`.

Neobrutalism for the dashboard

Thick black borders, hard (non-blurred) offset shadows, and saturated flat colors instead of gradients or soft elevation. For an operations dashboard — where the job is scanning many small numbers and status badges quickly — high contrast and unambiguous edges read faster than a softer, more decorative system would. It also happens to be a deliberate, opinionated choice rather than a generic component library default, which was the explicit brief.

Deployment Guide

[View on GitHub: docs/Deployment.md](#)

Deployment

Local development (no Docker)

Requirements: Python 3.12, Node 20+, PostgreSQL 16, Redis 7.

```
# Backend
cd backend
python3.12 -m venv .venv && source .venv/bin/activate
pip install -e ".[dev]"
cp .env.example .env # edit DATABASE_URL / REDIS_URL / JWT_SECRET_KEY if needed
alembic upgrade head
uvicorn app.main:app --reload --port 8000

# In separate terminals:
python -m app.scheduler.main
WORKER_PROJECT_ID=<uuid-of-a-project> python -m app.workers.main

# Frontend
cd frontend
npm install
npm run dev # http://localhost:5173, proxies /api to :8000
```

Run tests: `cd backend && python -m pytest tests/ -v` (needs a `scheduler_test` Postgres database — see `tests/conftest.py`).

Docker Compose

```
cp .env.example .env # set a real JWT_SECRET_KEY
docker compose up --build
```

This starts `postgres`, `redis`, `api` (runs migrations on boot, then serves on :8000), `scheduler`, `frontend` (nginx on :8080, proxying `/api` to the `api` service), and `prometheus` (:9090).

The `worker` service is not started by default — it's declared with `profiles: ["worker"]` because a worker is scoped to one project, and no project exists until you've registered a user through the UI and created one. Bootstrap order:

```
docker compose up --build # everything except workers
open http://localhost:8080 # register, create a project, copy its id
WORKER_PROJECT_ID=<that-uuid> docker compose --profile worker up --scale worker=3 worker
```

Scaling workers horizontally is just `--scale worker=N` — they coordinate purely through Postgres row locks (`SKIP LOCKED`), so there's no coordinator to configure.

Environment variables

All backend config is in `backend/app/config/settings.py` (pydantic-settings, reads `.env`). The ones you're likely to change in production:

Variable	Default	Notes
<code>DATABASE_URL</code>	<code>postgresql+asyncpg://scheduler:scheduler@localhost:5432/scheduler</code>	Must use the <code>asyncpg</code> driver
<code>REDIS_URL</code>	<code>redis://localhost:6379/0</code>	

Variable	Default	Notes
<code>JWT_SECRET_KEY</code>	dev placeholder	Generate with <code>openssl rand -hex 32</code> ; never reuse the default.
<code>ACCESS_TOKEN_EXPIRE_MINUTES</code> / <code>REFRESH_TOKEN_EXPIRE_DAYS</code>	30 / 7	
<code>CORS_ORIGINS</code>	localhost dev ports	Comma-separated
<code>WORKER_CONCURRENCY</code>	10	Max concurrent jobs per worker process
<code>WORKER_POLL_INTERVAL_MS</code>	500	Fallback poll interval when no Redis Stream notification arrives
<code>WORKER_HEARTBEAT_INTERVAL_SEC</code> / <code>WORKER_DRAIN_TIMEOUT_SEC</code>	5 / 30	Graceful shutdown waits up to the drain timeout for in-flight jobs
<code>SCHEDULER_SCAN_INTERVAL_SEC</code>	1	How often the delay/cron scanners run
<code>SCHEDULER_LEADER_LOCK_TTL_SEC</code>	15	Redis-lock TTL for scheduler leader election (safe to run multiple scheduler replicas)

Variable	Default	Notes
<code>RATE_LIMIT_REQUESTS</code> / <code>RATE_LIMIT_WINDOW_SEC</code>	100 / 60	Fixed-window limit per client IP

Migrations

`alembic upgrade head` (run automatically by the API container's entrypoint; run manually for local dev). New migration: `alembic revision --autogenerate -m "description"` after changing models in `app/models/`.

Production considerations not yet wired up

Being upfront about what's demonstrated vs. what a real production rollout still needs:

- **TLS termination** — put the API and frontend behind a real load balancer / ingress (nginx, Traefik, or a cloud LB) with certificates; nothing here terminates TLS itself.
- **Secrets management** — `JWT_SECRET_KEY` and the Postgres password are plain env vars here; use your platform's secret store (Vault, AWS Secrets Manager, k8s Secrets) instead.
- **Postgres HA** — this compose file runs a single Postgres instance. Production needs replication and backups (e.g. managed RDS/Cloud SQL, or Patroni).
- **Redis persistence** — Redis here is a plain cache/notification bus and can be lost without data loss, but a Sentinel/Cluster setup avoids the brief claim-latency blip a Redis restart causes.
- **Log/metric shipping** — structured JSON logs go to stdout (ready for any log collector); Prometheus is wired up for the API only — worker/scheduler processes don't yet expose a `/metrics` port.

Generated automatically from the project's own documentation sources (README.md, docs/*.md) in <https://github.com/moneshs23/Job-Scheduler>.
Diagrams rendered live from Mermaid source — not screenshots — so they stay in sync with the source of truth.